InternNova

6 Week Java Development Internship

WEEK 6 - FINAL PROJECT

BANK ACCOUNT MANAGEMENT SYSTEM

(Console Based Java Application)

Submitted by

V. RAMASHANMUGI

III Year B.E. CSE (AI & ML)

Register No: 923824148044

Mangayarkarasi College of Engineering

GitHub: https://github.com/ramashanmugi-v/InternNova-Week6-Assignment

1. Project Title

Bank Account Management System

2. Introduction

This project is a bank account management system made using Java. It runs in the console and works with a menu. Using this application we can create a bank account, see all accounts, search an account, change the account holder name, deposit money, withdraw money and delete an account.

There are two types of accounts in this project. One is Savings account and the other is Current account. In Savings account the balance cannot go below Rs.500. In Current account we can withdraw more than the balance up to Rs.5000 (overdraft).

All the account details are saved in a text file called accounts.txt. So even if we close the program and open it again, the data will still be there.

3. Objective

The main aim of this project is to use the Java topics which I learned in the 6 week internship and make one full working application. The other objectives are:

- To use OOP concepts like classes, inheritance, polymorphism, encapsulation and abstraction.
- To handle errors using exception handling so that the program does not stop suddenly.
- To use collections like ArrayList, HashSet and HashMap.
- To save and read data using file handling.
- To use multithreading for the auto save feature.
- To learn Git and GitHub by uploading the project in a repository.

4. Features

- Create Account - Savings or Current. Account number is given automatically from 1001.
- View All Accounts - shows all accounts in a table.
- Search Account - search using account number.
- Update Account - change the account holder name.
- Deposit - add money to an account.
- Withdraw - take money from an account with balance rules.
- Delete Account - remove an account.
- Transaction History - shows the operations done in the current run.
- File storage - data is saved in accounts.txt and loaded again when the program starts.
- Auto save - a background thread saves the data every 10 seconds.

- Input checking - wrong input like letters or negative amount will not crash the program.

5. Concepts Used

Concept	Where it is used in the project
Classes and Objects	Account, SavingsAccount, CurrentAccount, BankService
Constructors and this	Constructor of Account class
static	nextAccountNumber in BankService, MIN_BALANCE in SavingsAccount
Inheritance	SavingsAccount and CurrentAccount extend Account
Polymorphism	getMinBalance() works differently in each account type
Encapsulation	Private variables with getters and setters in Account
Abstraction	Abstract class Account and interface Transactable
Exception Handling	Custom exceptions, try-catch, try-with-resources
Collections	HashMap for searching, HashSet for no duplicate account numbers, ArrayList for history
File Handling	FileHandler class using FileReader, FileWriter, BufferedReader, PrintWriter
Multithreading	AutoSaveThread class and synchronized methods
Git and GitHub	Repository, commits, .gitignore, push

6. Project Structure

The project has 11 Java files inside the src folder.

```
BankSystem
   src
      Main.java
      Transactable.java  (interface)
      Account.java  (abstract class)
      SavingsAccount.java
      CurrentAccount.java
      BankService.java
      FileHandler.java
      AutoSaveThread.java
      InvalidAmountException.java
      InsufficientBalanceException.java
      AccountNotFoundException.java
      accounts.txt
   Screenshots
   README.md
   .gitignore
```

Account is the abstract class and it implements the Transactable interface. SavingsAccount and CurrentAccount extend the Account class. BankService keeps all the accounts in a HashMap. FileHandler saves and loads the data and AutoSaveThread calls it in the background. Main class shows the menu.

7. Code Screenshots

7.1 Account.java

This is the abstract class. It has private variables, a constructor with this keyword and the withdraw method which throws custom exceptions.

Account.java

```java
// Abstract class: common data and behaviour for all account types
public abstract class Account implements Transactable {
    private final int accountNumber;      // Encapsulation: private fields
    private String holderName;
    protected double balance;

    public Account(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;   // 'this' keyword
        this.holderName = holderName;
        this.balance = balance;
    }

    public abstract String getAccountType();
    public abstract double getMinBalance();   // rule differs per account type

    public int getAccountNumber() { return accountNumber; }
    public String getHolderName() { return holderName; }
    public double getBalance() { return balance; }
    public void setHolderName(String holderName) { this.holderName = holderName; }

    @Override
    public void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Deposit amount must be greater than 0.");
        }
        balance += amount;
    }

    @Override
    public void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException {
        if (amount <= 0) {
            throw new InvalidAmountException("Withdrawal amount must be greater than 0.");
        }
        if (balance - amount < getMinBalance()) {
            throw new InsufficientBalanceException("Insufficient balance. Available: Rs."
                    + String.format("%.2f", balance - getMinBalance()));
        }
        balance -= amount;
    }

    public String toFileString() {
        return getAccountType() + "," + accountNumber + "," + holderName + "," + balance;
    }

    @Override
    public String toString() {
        return String.format("%-8d %-20s %-10s Rs.%.2f", accountNumber, holderName, getAccountType(), balance);
    }
}
```

Fig 7.1 Account.java

7.2 Transactable.java (interface) and custom exception

Transactable.java

```java
public interface Transactable {
    void deposit(double amount) throws InvalidAmountException;
    void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException;
}
```

Fig 7.2 Transactable.java

InsufficientBalanceException.java

```java
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

Fig 7.3 InsufficientBalanceException.java

7.3 SavingsAccount.java and CurrentAccount.java

Both classes extend Account and give different minimum balance. This is polymorphism.

SavingsAccount.java

```java
// Inheritance: SavingsAccount IS-A Account
public class SavingsAccount extends Account {
    public static final double MIN_BALANCE = 500.0;   // static constant

    public SavingsAccount(int accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }

    @Override
    public String getAccountType() { return "SAVINGS"; }

    @Override
    public double getMinBalance() { return MIN_BALANCE; }   // Polymorphism
}
```

Fig 7.4 SavingsAccount.java

CurrentAccount.java

```java
public class CurrentAccount extends Account {
    public static final double OVERDRAFT_LIMIT = 5000.0;

    public CurrentAccount(int accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }

    @Override
    public String getAccountType() { return "CURRENT"; }

    @Override
    public double getMinBalance() { return -OVERDRAFT_LIMIT; }   // Polymorphism
}
```

Fig 7.5 CurrentAccount.java

7.4 BankService.java

Here HashMap, HashSet and ArrayList are used. nextAccountNumber is a static variable.

BankService.java

```java
import java.util.*;

public class BankService {
    private final Map<Integer, Account> accounts = new HashMap<>();   // fast search by account number
    private final Set<Integer> usedNumbers = new HashSet<>();         // no duplicate account numbers
    private final List<String> transactionLog = new ArrayList<>();    // history
    private static int nextAccountNumber = 1001;                      // static counter

    public synchronized Account createAccount(String type, String name, double initial)
            throws InvalidAmountException {
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidAmountException("Name cannot be empty.");
        }
        name = name.trim().replace(",", " ");
        Account acc;
        if (type.equals("SAVINGS")) {
            if (initial < SavingsAccount.MIN_BALANCE) {
                throw new InvalidAmountException("Savings account needs minimum Rs."
                        + SavingsAccount.MIN_BALANCE + " opening balance.");
            }
            acc = new SavingsAccount(nextAccountNumber, name, initial);
        } else {
            if (initial < 0) {
                throw new InvalidAmountException("Opening balance cannot be negative.");
            }
            acc = new CurrentAccount(nextAccountNumber, name, initial);
        }
        usedNumbers.add(nextAccountNumber);
        accounts.put(nextAccountNumber, acc);
        nextAccountNumber++;
        transactionLog.add("Created account " + acc.getAccountNumber());
        return acc;
    }
```

Fig 7.6 BankService.java

7.5 FileHandler.java

This class saves the accounts to accounts.txt and loads them back.

FileHandler.java

```java
import java.io.*;
import java.util.*;

public class FileHandler {

    public static void save(List<Account> accounts, String path) {
        try (PrintWriter out = new PrintWriter(new FileWriter(path))) {
            for (Account a : accounts) {
                out.println(a.toFileString());
            }
        } catch (IOException e) {
            System.out.println("Error saving file: " + e.getMessage());
        }
    }

    public static List<Account> load(String path) {
        List<Account> list = new ArrayList<>();
        File file = new File(path);
        if (!file.exists()) return list;   // first run: no file yet

        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] p = line.split(",");
                if (p.length != 4) continue;
                try {
                    int no = Integer.parseInt(p[1].trim());
                    double bal = Double.parseDouble(p[3].trim());
                    if (p[0].equals("SAVINGS")) {
                        list.add(new SavingsAccount(no, p[2], bal));
                    } else if (p[0].equals("CURRENT")) {
                        list.add(new CurrentAccount(no, p[2], bal));
                    }
                } catch (NumberFormatException e) {
                    System.out.println("Skipping corrupted line: " + line);
                }
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
        return list;
    }
}
```

Fig 7.7 FileHandler.java

7.6 AutoSaveThread.java

This thread runs in the background and saves the data every 10 seconds.

AutoSaveThread.java

```java
// Multithreading: saves data in background every few seconds
public class AutoSaveThread extends Thread {
    private final BankService service;
    private final String path;
    private volatile boolean running = true;

    public AutoSaveThread(BankService service, String path) {
        this.service = service;
        this.path = path;
        setDaemon(true);   // stops automatically when program exits
    }

    public void stopSaving() {
        running = false;
        interrupt();
    }

    @Override
    public void run() {
        while (running) {
            try {
                Thread.sleep(10000);   // every 10 seconds
            } catch (InterruptedException e) {
                break;
            }
            FileHandler.save(service.getAll(), path);
        }
    }
}
```

Fig 7.8 AutoSaveThread.java

7.7 Main.java

The main method loads the data, starts the auto save thread and shows the menu in a loop.

Main.java

```java
import java.util.*;

public class Main {
    private static final String DATA_FILE = "accounts.txt";
    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        BankService service = new BankService();
        for (Account a : FileHandler.load(DATA_FILE)) {
            service.addLoaded(a);
        }

        AutoSaveThread autoSave = new AutoSaveThread(service, DATA_FILE);
        autoSave.start();

        boolean running = true;
        while (running) {
            printMenu();
            int choice = readInt("Enter choice: ");
            try {
                switch (choice) {
                    case 1: createAccount(service); break;
                    case 2: viewAll(service); break;
                    case 3: searchAccount(service); break;
                    case 4: updateAccount(service); break;
                    case 5: deposit(service); break;
                    case 6: withdraw(service); break;
                    case 7: deleteAccount(service); break;
                    case 8: showHistory(service); break;
                    case 9:
                        running = false;
                        break;
                    default:
                        System.out.println("Invalid choice. Enter 1-9.");
                }
            } catch (AccountNotFoundException | InvalidAmountException | InsufficientBalanceException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }

        autoSave.stopSaving();
        FileHandler.save(service.getAll(), DATA_FILE);
        System.out.println("Data saved. Thank you for using the Bank System!");
    }
```

Fig 7.9 Main.java (main method)

8. Output Screenshots

The screenshots below show the output of the project. First an account is created, then searched, updated, deposit and withdraw are done and finally the account is deleted.

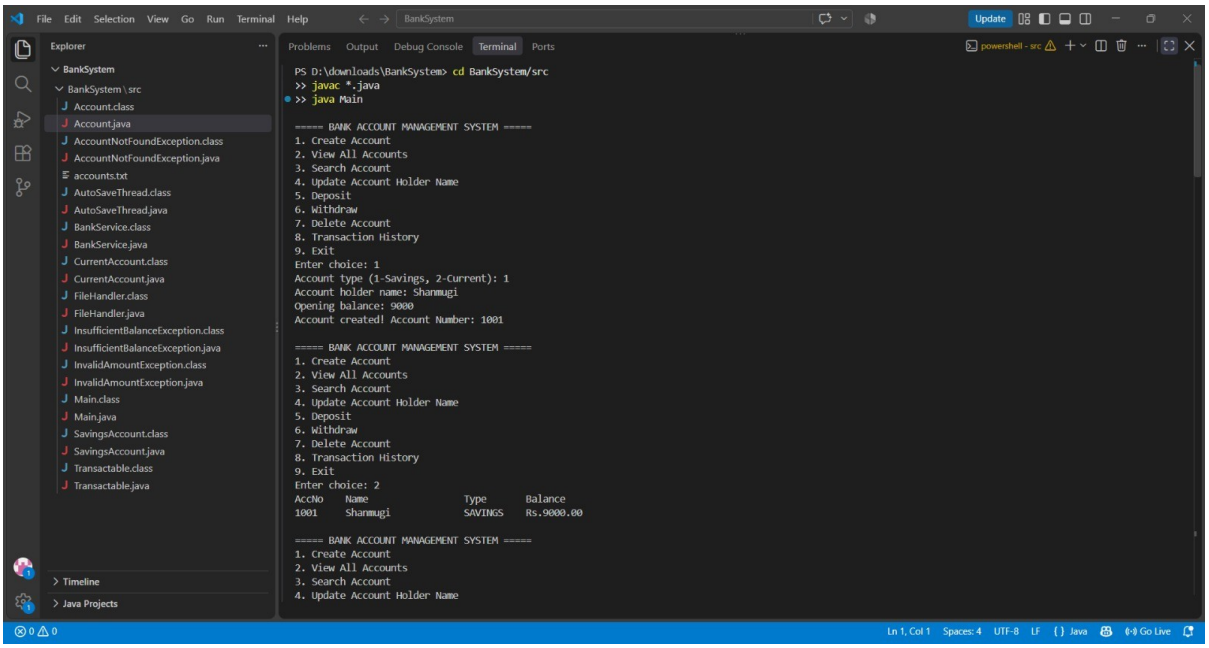


Fig 8.1 Create Account and View All Accounts

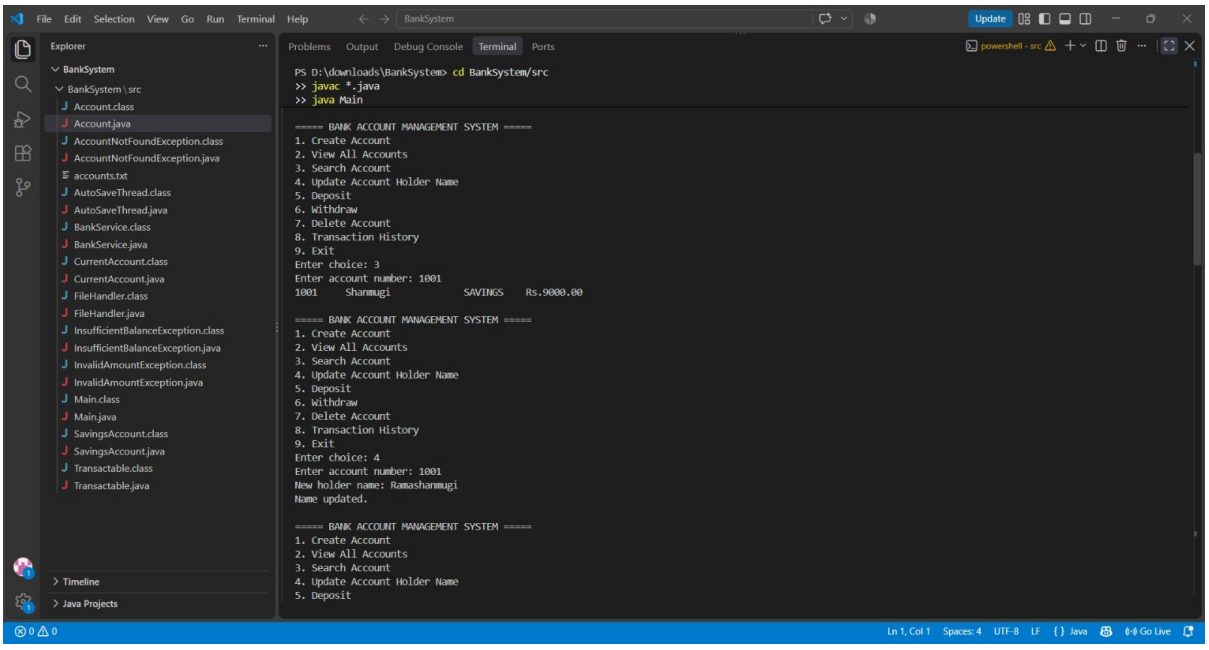


Fig 8.2 Search Account and Update Account Holder Name

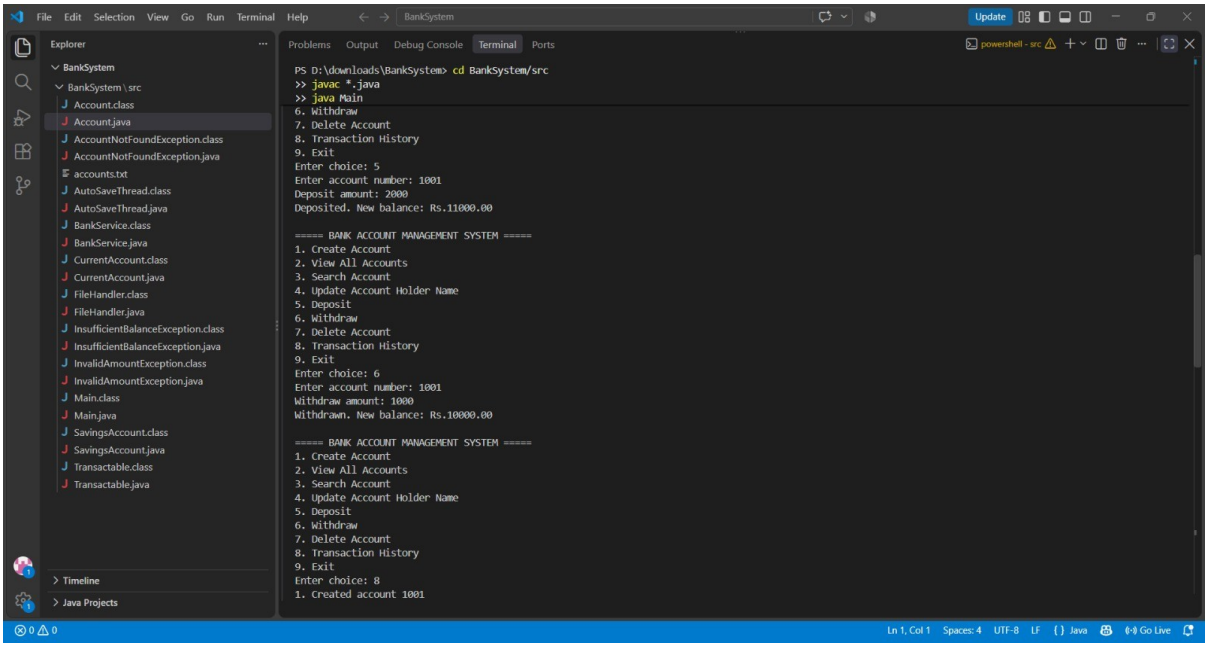


Fig 8.3 Deposit and Withdraw

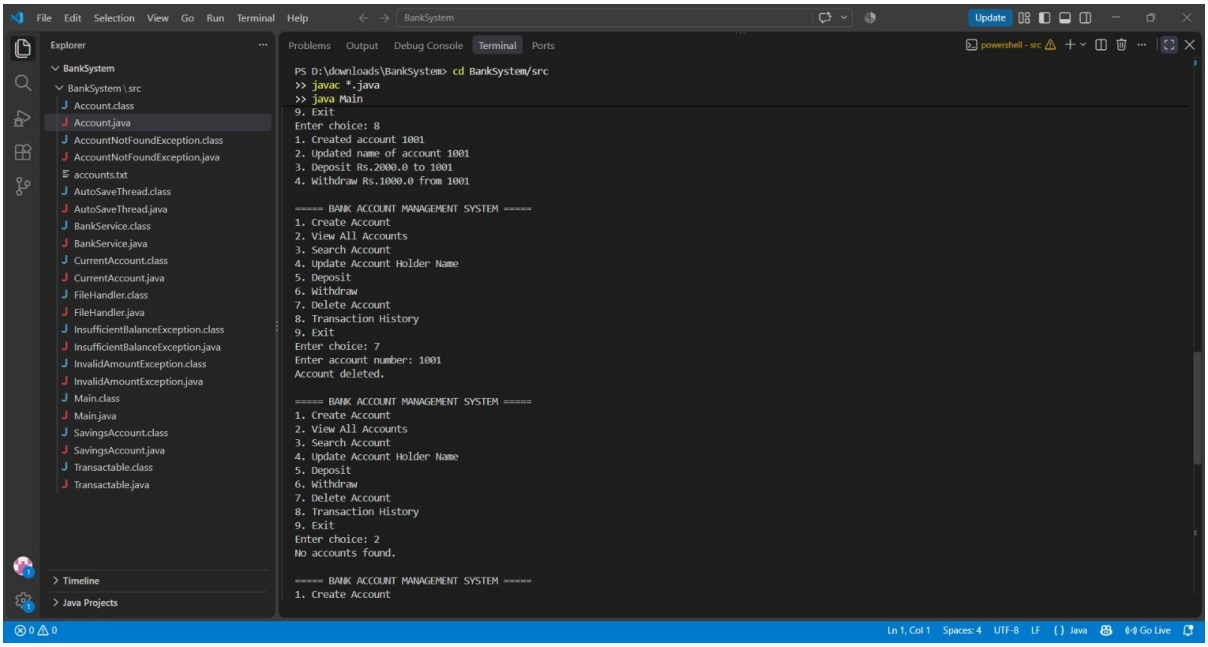


Fig 8.4 Transaction History and Delete Account

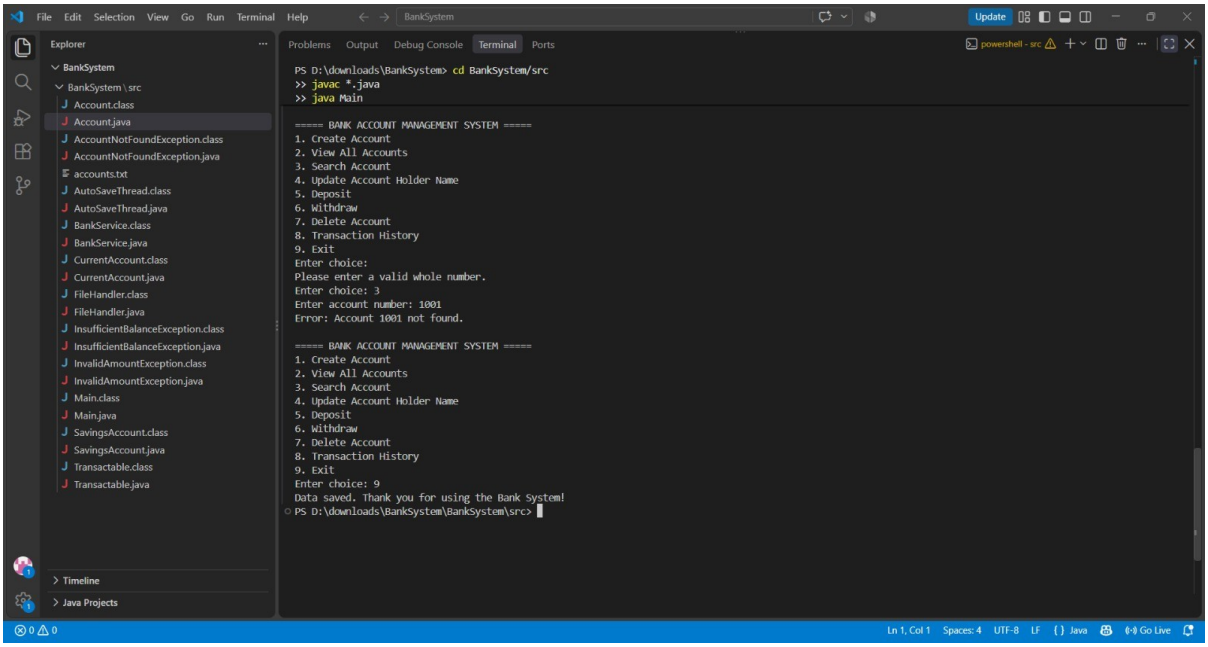


Fig 8.5 Wrong input, Account not found error and Exit

9. Challenges Faced

1. Different rules for two account types. Savings account needs minimum balance but Current account allows overdraft. I solved this by making getMinBalance() an abstract method in Account and overriding it in both child classes.

2. The program was stopping when I typed letters instead of numbers. I solved this by writing readInt() and readDouble() methods which use try-catch and ask the user to enter the value again.

3. Data was lost when the program was closed. I solved this by using file handling to save the accounts in accounts.txt and load them when the program starts.

4. Git push was rejected the first time because the GitHub repository already had a README file. I solved this by using git pull with --allow-unrelated-histories and then pushed again.

5. The .class files also went to GitHub. I removed them using git rm --cached and added a .gitignore file with *.class.

10. Conclusion

By doing this project I understood how to use all the Java topics together in one application. I learned how abstract class, interface and inheritance help to reduce code, how custom exceptions show proper error messages, how HashMap and HashSet are useful, and how file handling saves data permanently. I also learned the basics of multithreading with the auto save feature.

I also learned to use Git and GitHub to create a repository, commit the files and push the project. In future this project can be improved by adding login with PIN, money transfer between accounts, interest calculation and a web version using Spring Boot.